Multi-Class Pokémon Classification System

A Comparative Study of CNN Architectures and Regularization Techniques

Junjie Gu Rujia Li Xiaohan Wang

CONTENTS

1 Research Background and Objectives

2 Data Preparation and Preprocessing

3 Modeling

4 Comment & Conclusion

5 Contribution Statement

1 Research Background and Objectives

Experimental Background:

This project is based on deep learning to conduct a classification study on multi-class images of 1000 Pokémon species. The data size reaches 26,539 images with diverse categories and unbalanced sample distribution. We combine multiple optimizers, learning rate strategies, and regularization methods to investigate the impact of these techniques on the model performance.

Experimental Objective:

To explore how to achieve better test accuracy and better generalization ability in multi-category image classification tasks by comparing different learning rate strategies.

2 Data Preparation and Preprocessing

1. data augmentation strategy:

Random rotation , horizontal shifting , zoom , and horizontal flipping

2. Generating labels:

Read each image file and record the corresponding category labels

3. Split the data into training, validation and test sets:

Training set: The data size of the training set is set to 128 and the image size is adjusted to (224, 224) Finally, 19,434 images are loaded, covering 1000 categories.

Validation set and test set: The image size and labelling pattern are kept the same as the training set. The validation set is loaded with 2,815 images; the test set is loaded with 4,290 images.

3 Modeling

Custom CNN Model：

1. core idea：

Using convolutional layers to automatically learn and extract hierarchical features from images, replacing hand-engineered feature design.

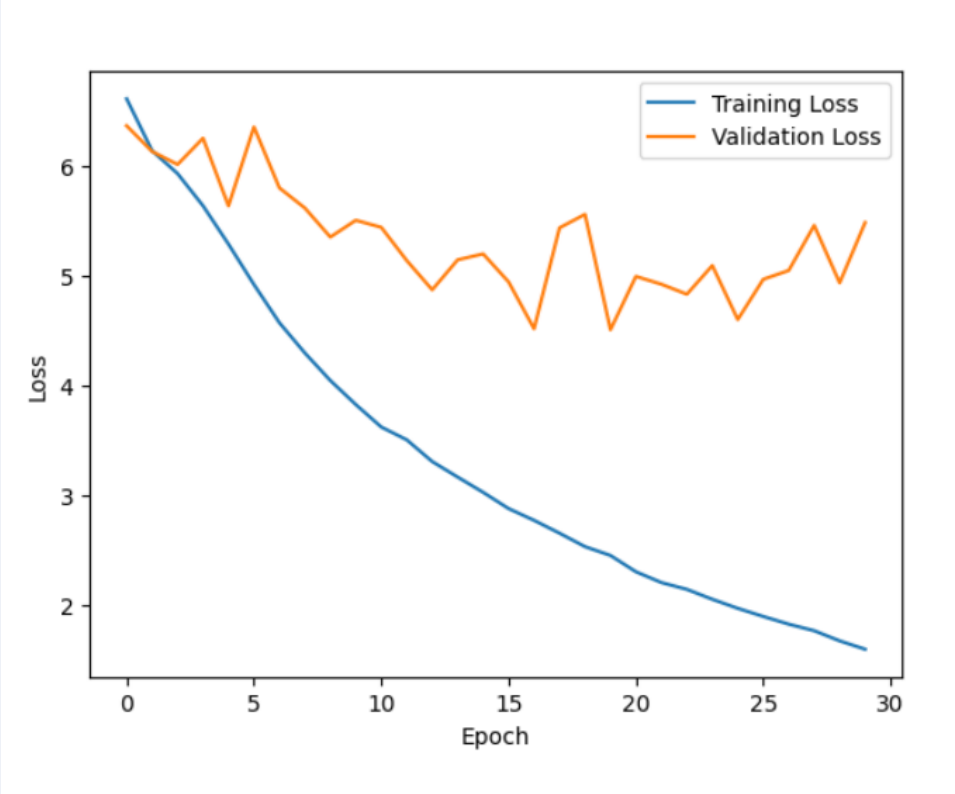
2. theory：

Convolutional Neural Networks (CNNs) use learnable filters over small regions of an image to capture basic patterns (e.g., edges) in early layers and more complex features (e.g., shapes) in deeper layers. Pooling operations reduce spatial dimensions while preserving key information. By stacking these layers, CNNs build abstract representations that deliver strong performance in tasks like image classification, all while reducing the number of parameters compared to fully connected networks and leveraging local connectivity for efficient training.

CNN Model

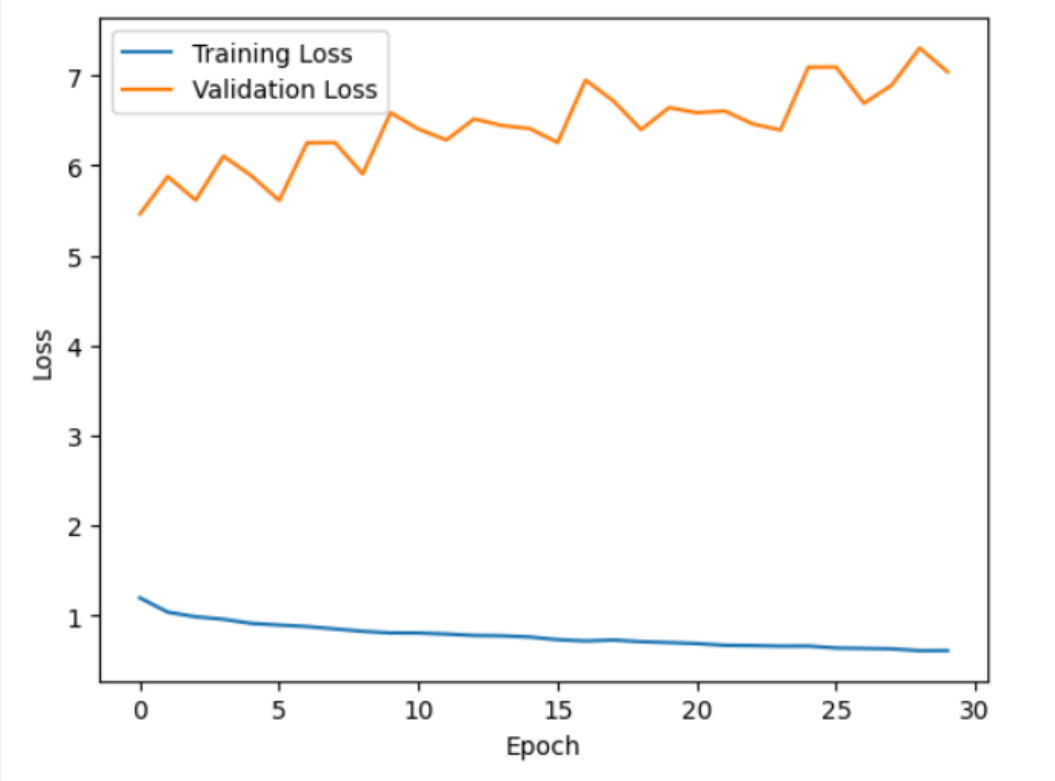
CNN1 Learning Rate = 0.001 Adam

Test Accuracy: 0.2212



CNN2 Learning Rate = 0.0001 Adam

Test Accuracy: 0.3028

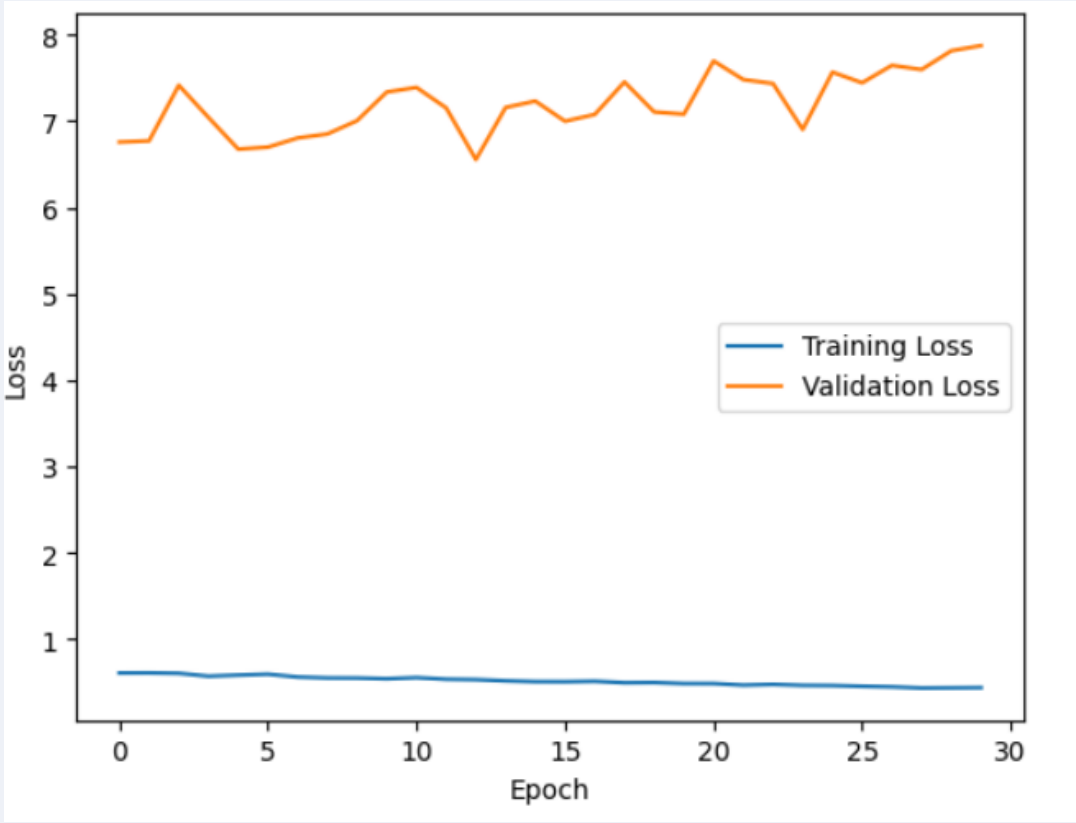


Loss curves for training and validation

CNN Model

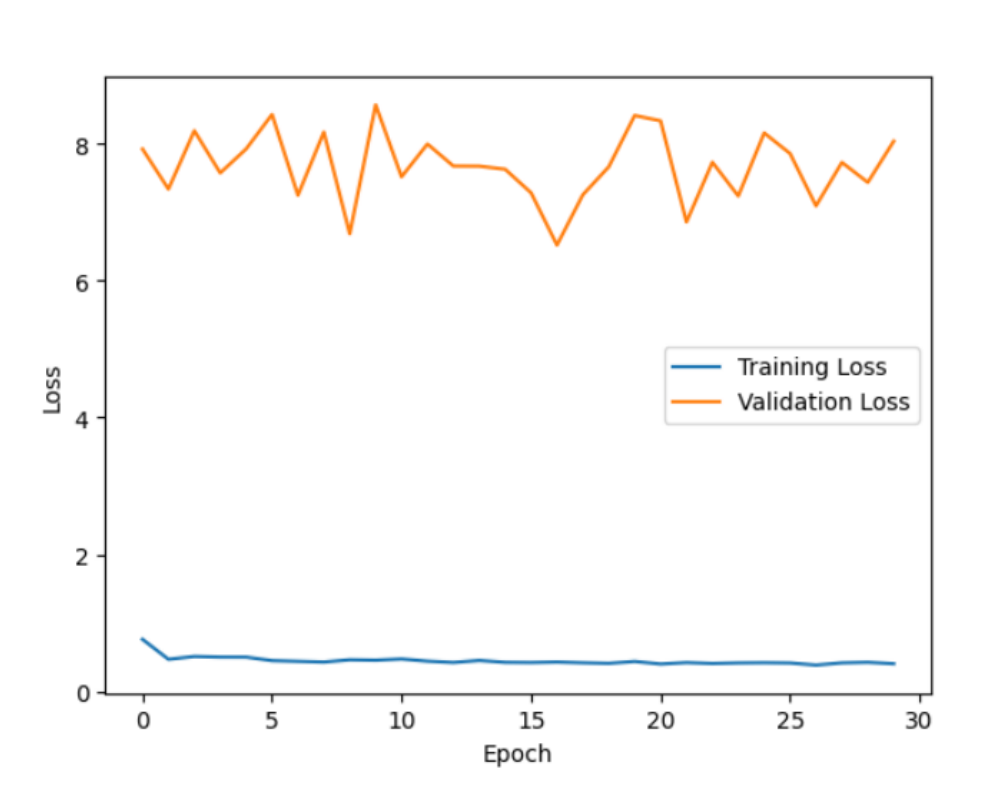
CNN3 Learning Rate = 0.0001 Nadam

Test Accuracy: 0.3166



CNN4 Learning Rate = 0.0001 SGD

Test Accuracy: 0.3245



Loss curves for training and validation

CNN Model：

Table 3: CNN Model Performance

Variation	Learning Rate	Optimizer	Test Accuracy (%)
CNN1	0.001	Adam	22.12
CNN2	0.0001	Adam	30.28
CNN3	0.0001	Nadam	31.66
CNN4	0.0001	SGD	32.45

1.Standard CNN architectures perform significantly worse than ResNet and MobileNet, with a maximum accuracy of only 32.45%.

2.Lower learning rates (0.0001) improve CNN performance, as seen in CNN2, CNN3, and CNN4.

MobileNetV2 Model：

1. core idea：

Using depthwise separable convolutions to drastically reduce computation while maintaining competitive accuracy.

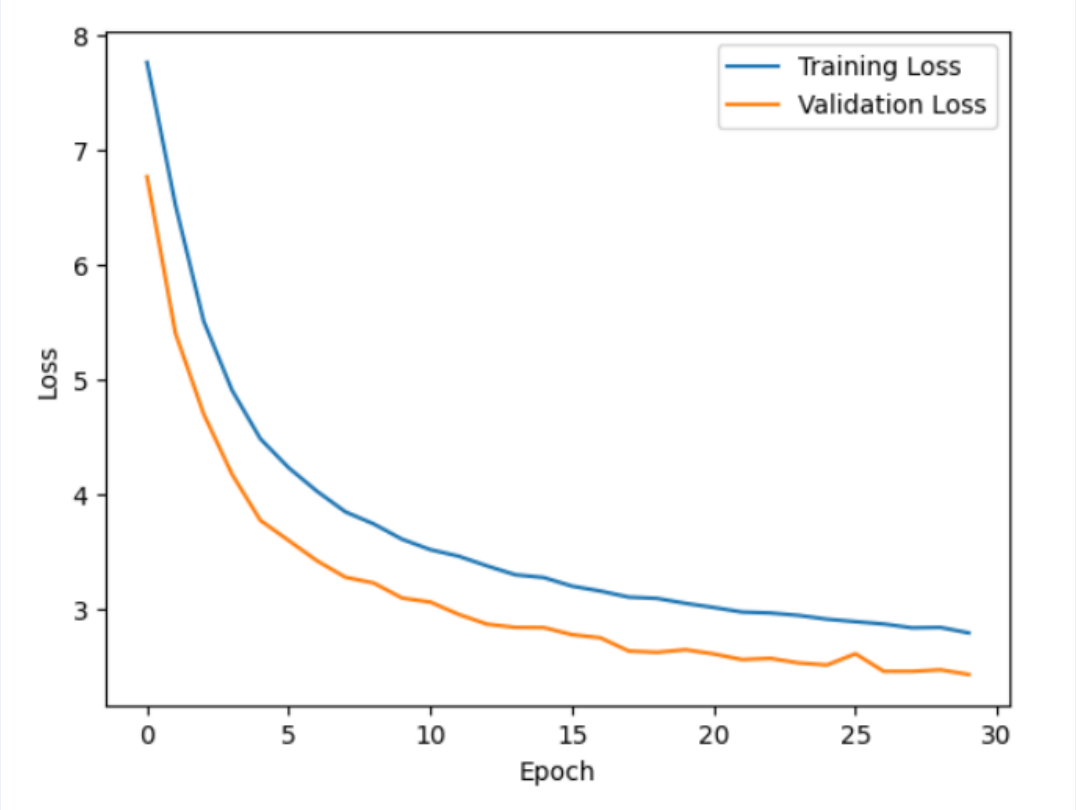
2. theory：

MobileNet decomposes a standard convolution into a depthwise step (filtering each input channel independently) and a pointwise step (using 1×1 filters to mix channels). This approach significantly cuts down the number of parameters and floating-point operations, making MobileNet highly efficient. Additionally, inverted residual blocks and linear bottlenecks further optimize feature extraction and preserve important information, enabling real-time performance on resource-constrained devices.

MobileNetV2 Model

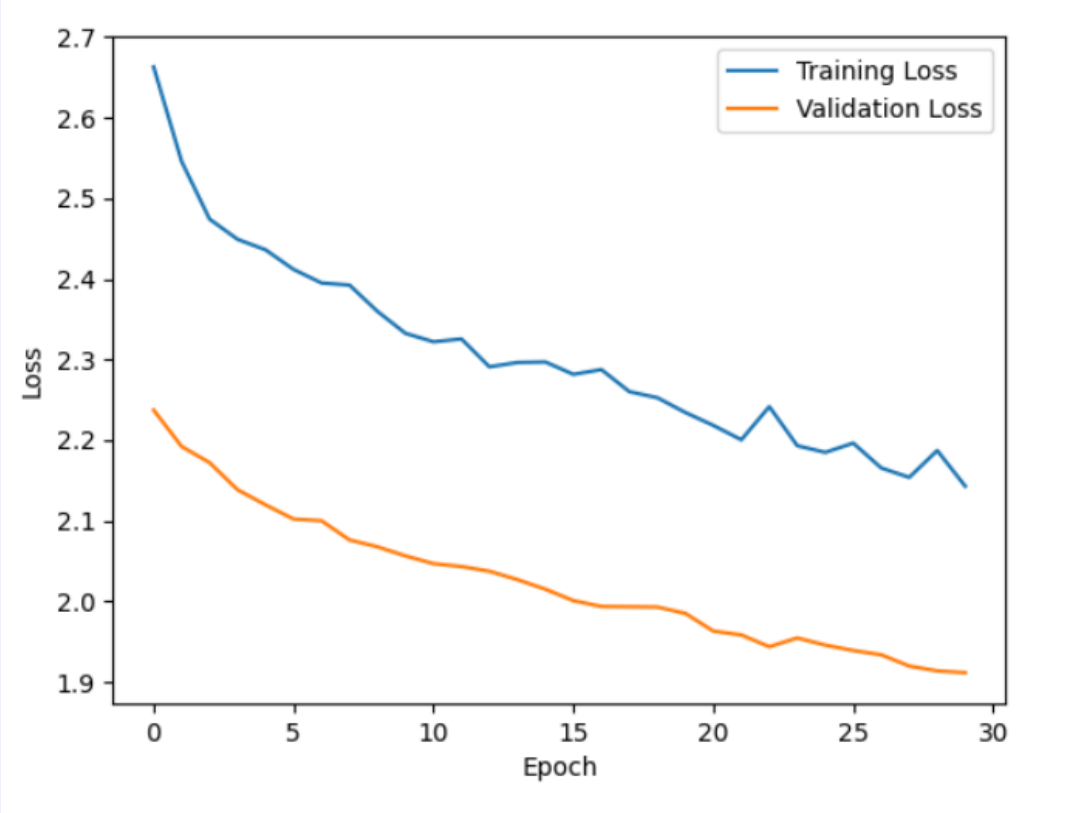
MoblieNet1 learning rate=0.001 Adam

Test Accuracy: 0.5739



MobileNet2 learning rate=0.0001 Adam

Test Accuracy: 0.6685

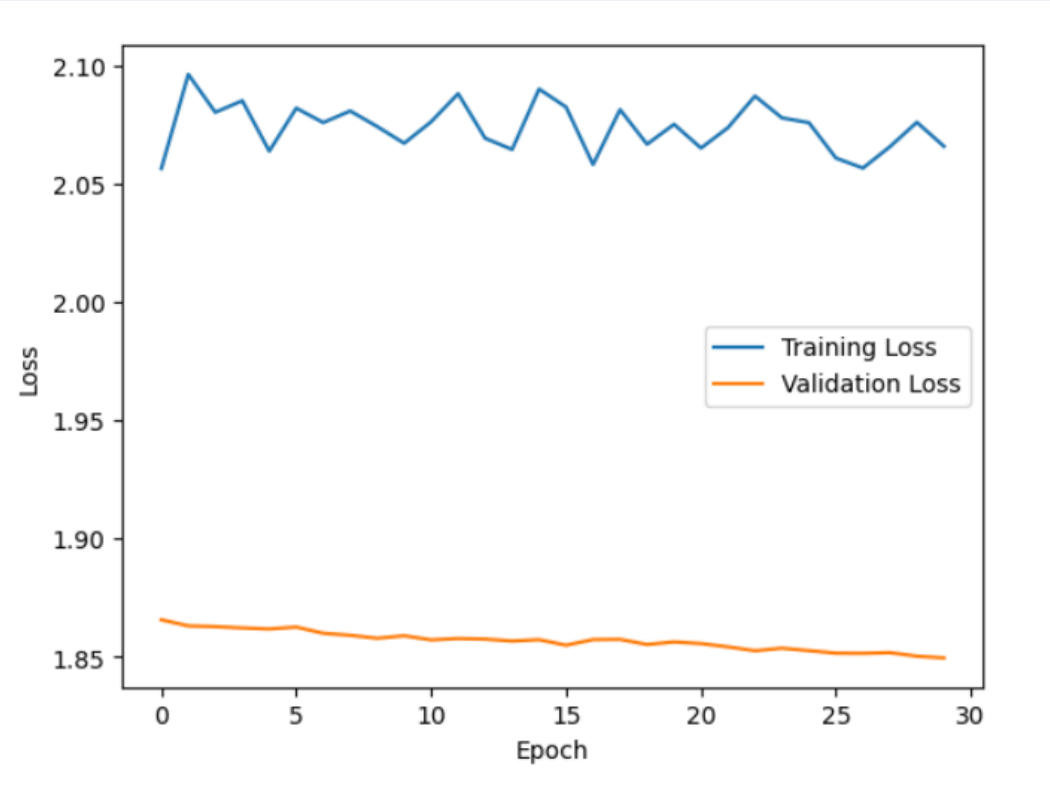


Loss curves for training and validation

MobileNetV2 Model

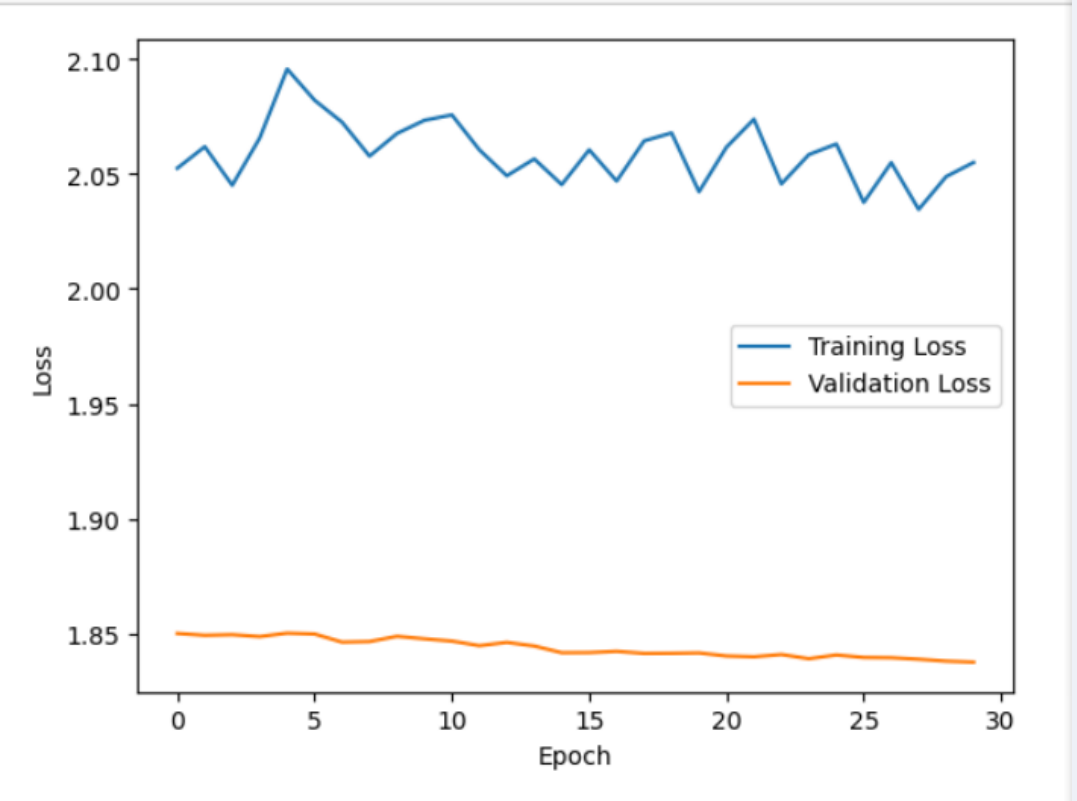
MobileNet3 learning rate=0.0001 Nadam

Test Accuracy: 0.6834



MobileNet4 learning rate=0.0001 SGD

Test Accuracy: 0.6858



Loss curves for training and validation

MobileNet Model：

Table 2: MobileNet Model Performance

Variation	Learning Rate	Optimizer	Test Accuracy (%)
MobileNet1	0.001	Adam	57.39
MobileNet2	0.0001	Adam	66.85
MobileNet3	0.0001	Nadam	68.34
MobileNet4	0.0001	SGD	68.58

1.MobileNet4 (SGD, LR=0.0001) achieves the highest accuracy (68.58%), showing that traditional optimizers can still be effective for lightweight architectures.

2.Nadam (68.34%) and Adam (66.85%) perform slightly lower than SGD.

3.Compared to ResNet, MobileNet has lower overall accuracy, but it is computationally efficient.

ResNet Model：

Learning From:

$$F(\vec{x}) = H(\vec{x}) - \vec{x}$$

If the Model is good enough:

$$F(\vec{x}) = 0$$

Residual module mapping：

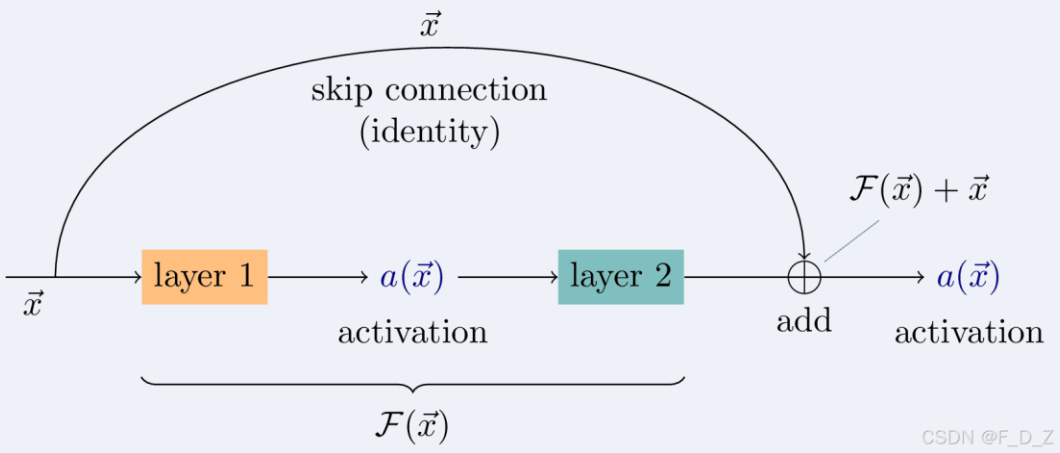


Image source: https://blog.csdn.net/a8039974/article/details/142202414

ResNet Model：

1. Base Model：

ResNet50

2. Parameters：

Total Parameters: 26,710,888

Trainable Parameters: 3,123,176

Non-Trainable Parameters: 23,587,712

3. Learning Rate：

$$Learning_rate(t) = initial_rate \times 0.96^{\left[\frac{t}{1000}\right]}$$

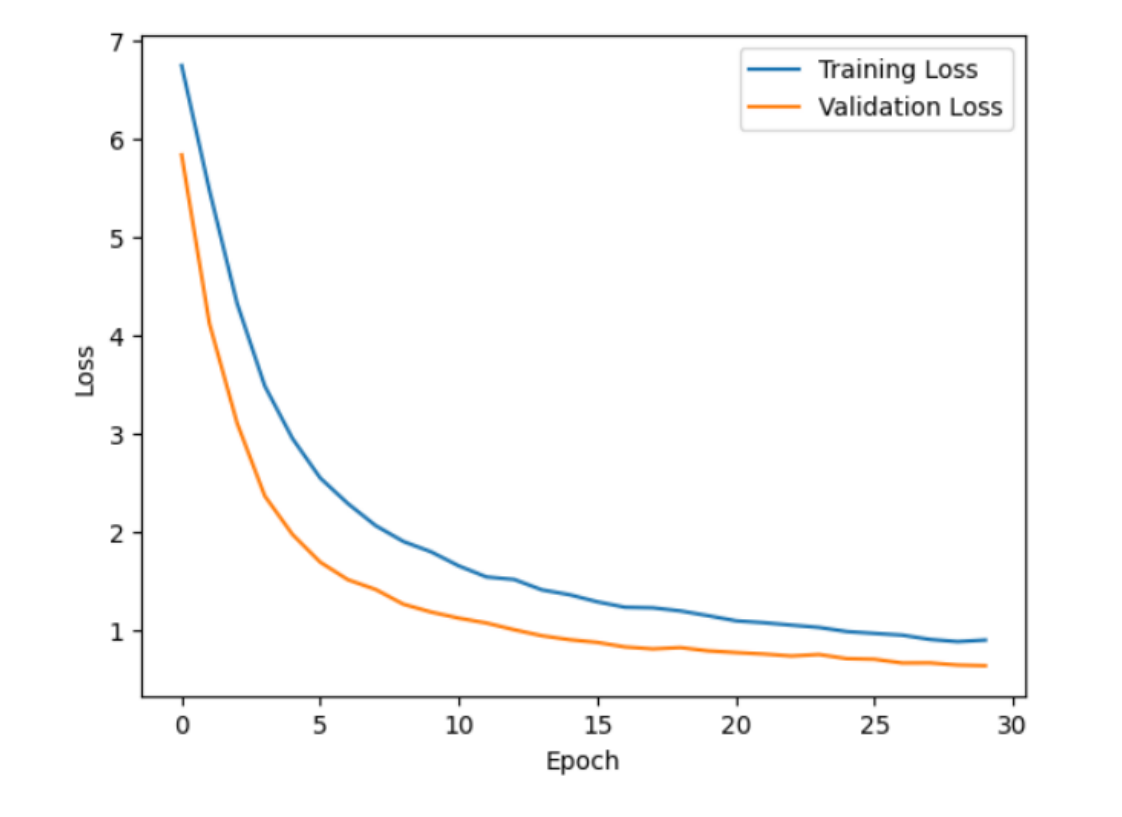
4. Inverse Class Frequency Weighting：

$$weight_i = \frac{N}{n_i \times C}$$

ResNet Model

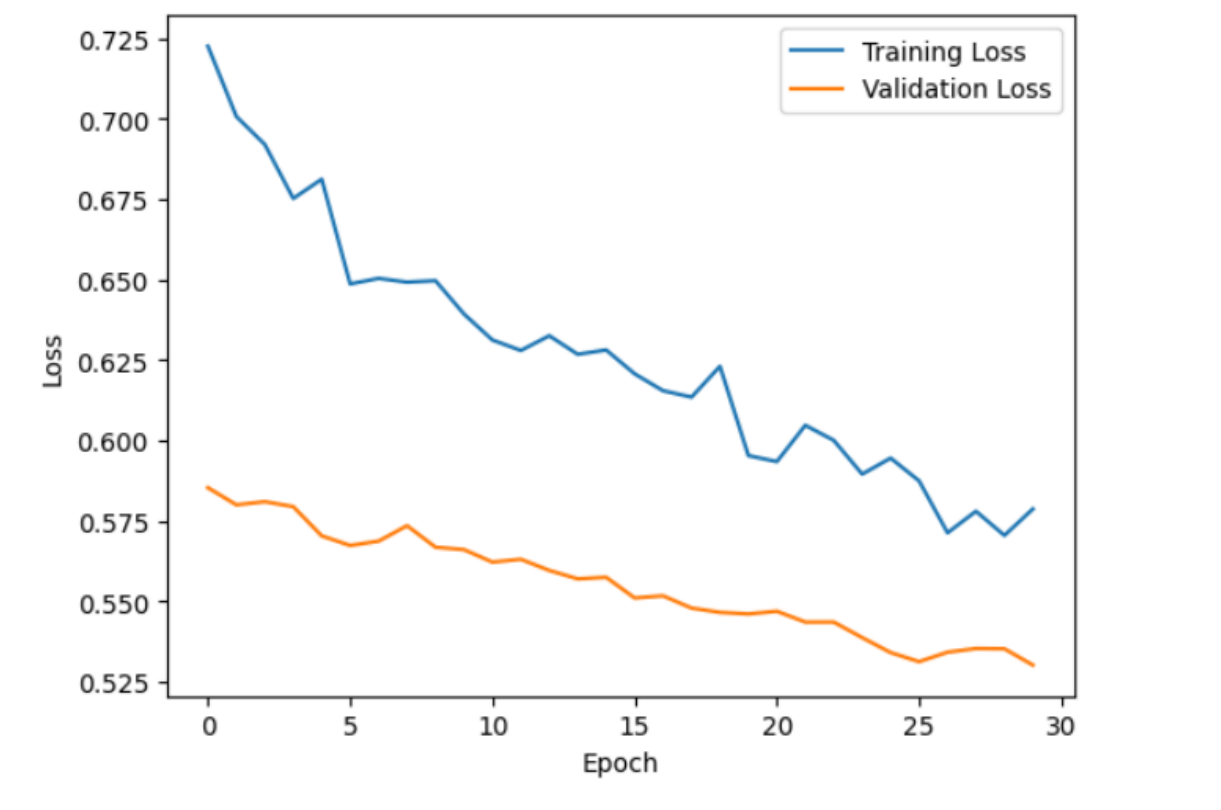
ResNet1 Learning Rate 0.001 Adam：

Test Accuracy: 0.8373



ResNet2 Learning Rate 0.0001 Adam：

Test Accuracy: 0.8636

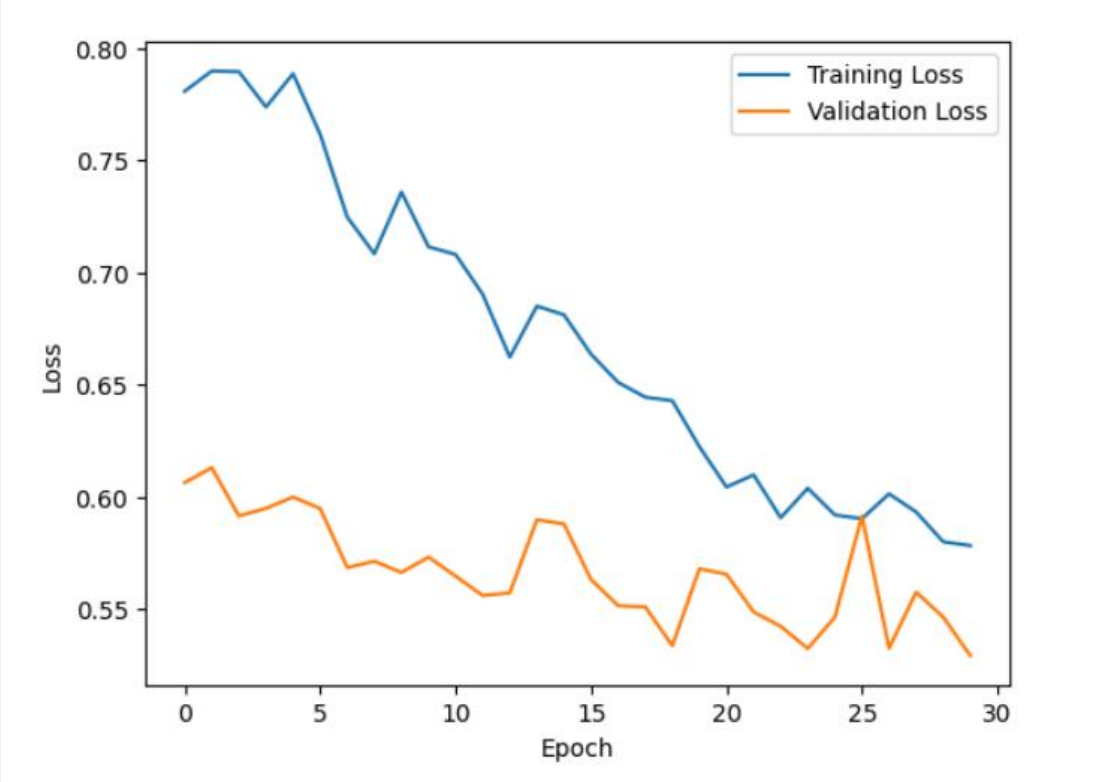


Loss curves for training and validation

ResNet Model

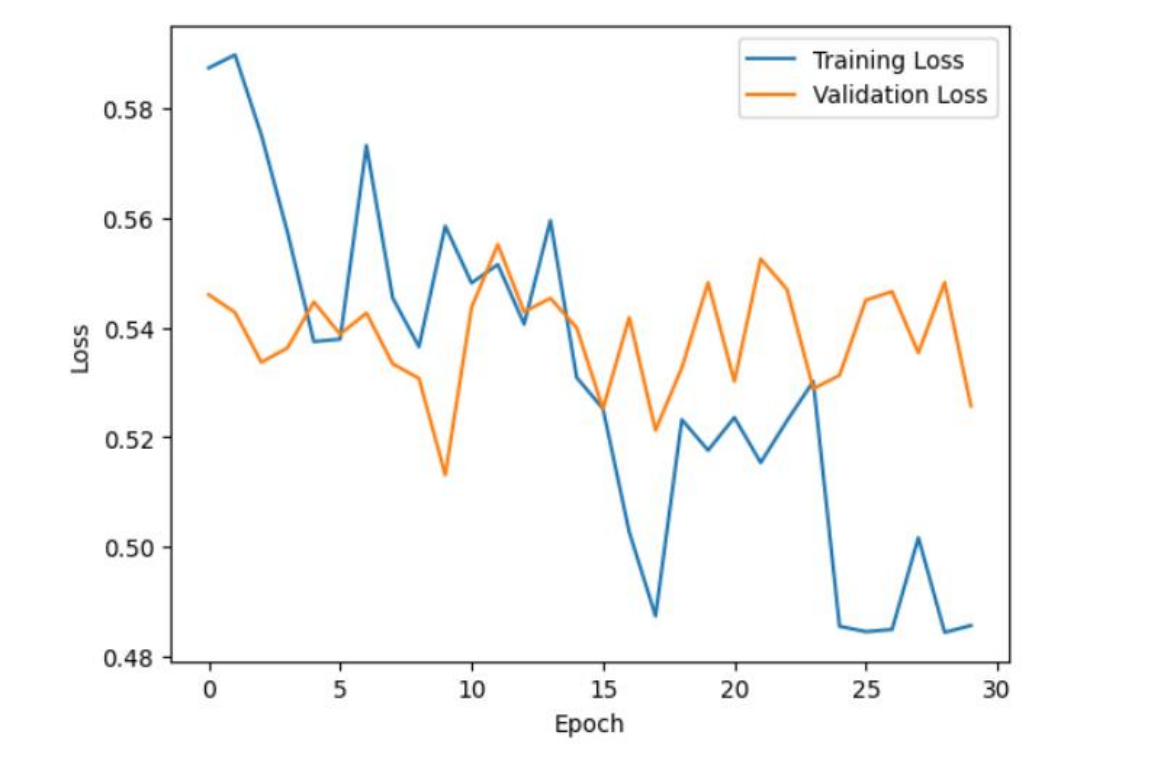
ResNet3 Learning Rate 0.0001 Nadam：

Test Accuracy: 0.8737



ResNet4 Learning Rate 0.0001 SGD：

Test Accuracy: 0.8718



Loss curves for training and validation

ResNet Model：

Table 1: ResNet Model Performance

Variation	Learning Rate	Optimizer	Test Accuracy (%)
ResNet1	0.001	Adam	83.73
ResNet2	0.0001	Adam	86.36
ResNet3	0.0001	Nadam	87.37
ResNet4	0.0001	SGD	87.18

1.Lower learning rate (0.0001) outperforms 0.001 across all cases, indicating that smaller step sizes enhance stability and generalization.

2.ResNet3 (Nadam) achieves the highest test accuracy (87.37%), suggesting that residual networks benefit from adaptive momentum-based optimizers.

4 Comment & Conclusion

Comment：

1.MobileNetV2 vs ResNet50:

Total_Parameters_MobileNetV2: 3,304,232

Total_Parameters_ResNet50: 26,710,888

2.Date enhancements:

Data enhancements are more complex (rotation, translation, etc.)

and can be a major time-consuming part of the project.

Conclusion：

ResNet50 shows strong feature extraction in Pokemon thousand classification task

Nadam+ lower stairstep decay learning rate is the best configuration.

MobileNetV2 balances between efficiency and performance.

Custom CNN needs structure optimization.

5 Contribution Statement

Contribution Statement：

Rujia Li : Choosing Dataset, Introduction, Custom CNN Model

Xiaohan Wang : MobileNet Model, Presentation Slides

Junjie Gu: ResNet Model, Combining code, Conclusion.

thank you!

Junjie Gu Rujia Li Xiaohan Wang